

Practice Interview

Objective

The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.

Group Size

Each group should have 2 people. You will be assigned a partner

Parts:

- Part 1: Complete 1 of 3 questions
- Part 2: Review your partner's Assignment 1 submission
- Part 3: Perform code review of your partner's assignment 1 by answering the questions below
- Part 3: Reflect on Assignment 1 and Assignment 2

Part 1:

You will be assigned one of three problems based on your first name. Enter your first name, in all lower case, execute the code below, and that will tell you your assigned problem. Include the output as part of your submission (do not clear the output). The problems are based-off problems from Leetcode.

```
In [3]: import hashlib

def hash_to_range(input_string: str) -> int:
    hash_object = hashlib.sha256(input_string.encode())
    hash_int = int(hash_object.hexdigest(), 16)
    return (hash_int % 3) + 1
input_string = "deeqa"
result = hash_to_range(input_string)
print(result)
```

2

► Question 1

Starter Code for Question 1

```
In [4]: # Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val = 0, left = None, right = None):
#         self.val = val
#         self.left = left
#         self.right = right
def is_duplicate(root: TreeNode) -> int:
    # TODO
```

Cell In[4], line 8

TODO

^

SyntaxError: incomplete input

► Question 2

Starter Code for Question 2

```
In [ ]: # Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val = 0, left = None, right = None):
#         self.val = val
#         self.left = left
#         self.right = right
```

```
In [5]: #bt_path(root: TreeNode) -> List[List[int]]:

def bt_path(root, lvl=0, current_path=None):
    if current_path == None:
        current_path = []

    tree_len = len(root)

    # Base case: Index out of bounds or node is missing
    if lvl >= tree_len or root[lvl] == None:
        return []

    # Add current node to the path
    current_path.append(root[lvl])

    left_child_index = 2 * lvl + 1
    right_child_index = 2 * lvl + 2

    # Check if this node is a leaf
    is_left_empty = left_child_index >= tree_len or root[left_child_index] == None
    is_right_empty = right_child_index >= tree_len or root[right_child_index] == None

    if is_left_empty and is_right_empty:

        result = [list(current_path)]

    else:

        result = bt_path(root, left_child_index, current_path) + \
            bt_path(root, right_child_index, current_path)
```

```
current_path.pop() #remove the current node before backtracking

return result
```

```
In [6]: root = [10,9,7,8]
        bt_path(root)
```

```
Out[6]: [[10, 9, 8], [10, 7]]
```

► Question 3

Starter Code for Question 3

```
In [ ]: def missing_num(nums: List) -> int:
        # TODO
```

Part 2:

You and your partner must share each other's Assignment 1 submission.

Part 3:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

```
In [ ]: #https://github.com/alf-99/algorithms\_and\_data\_structures/pull
```

- Copy the solution your partner wrote.

```
In [7]: from typing import List

def first_duplicate(nums: List[int]) -> int:
    seen = set()
    for num in nums:
        if num in seen:
            return num
        seen.add(num)
    return -1
```

- Create 1 new example that demonstrates you understand the problem.
Trace/walkthrough 1 example that your partner made and explain it.

```
In [8]: first_duplicate([1, 2, 3, 4, 5, 2, 1])
```

Out[8]: 2

```
In [9]: #Their example:
nums = [3, 1, 4, 2, 5, 1, 6]
first_duplicate(nums)
```

Out[9]: 1

```
In [10]: #The code first creates an empty set called "seen". It then iterates through each n
# Next is 1, which is not in "seen", so it is added.
# Then 4, which is also not in "seen", is added.
# Next is 2, which is not in "seen", so it is added.
# Then 5, which is not in "seen", is added.
# Next is 1 again, which is already in "seen", so the function returns 1 as the fir
# The number 6 is not evaluated because the function has already returned.
```

- Explain why their solution works in your own words.

```
In [11]: #Their code works by iterating through the list of numbers and keeping track of the
```

- Explain the problem's time and space complexity in your own words.

```
In [12]: # The time complexity of this code is O(n) because in the worst case, it needs to i
# The space complexity is also O(n) in the worst case, if there are no duplicates a
```

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

```
In [13]: # The code is very optimized and efficient for finding the first duplicate in a lis
```

Part 4:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

Reflection

```
In [14]: # My partner's explanation of the code is clear and concise. They correctly des
# Overall, I think their explanation is accurate and demonstrates a good unders
# I now realize code that is written with clear explanations and comments can be
```

Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

Submission Information

🔔 **Please review our [Assignment Submission Guide](#)** 🔔 for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

Submission Parameters:

- Submission Due Date: HH:MM AM/PM - DD/MM/YYYY
- The branch name for your repo should be: `assignment-2`
- What to submit for this assignment:
 - This Jupyter Notebook (`assignment_2.ipynb`) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment:
 - `https://github.com/<your_github_username>/algorithms_and_data_structures/`
 - Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- ☐ Created a branch with the correct naming convention.
- ☐ Ensured that the repository is public.
- ☐ Reviewed the PR description guidelines and adhered to them.
- ☐ Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at `#cohort-6-help`. Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.



In []: